

O que são testes parametrizados?

São testes que recebem **vários conjuntos de dados** automaticamente.

👉 Em vez de criar vários métodos:

```
@Test
void teste1() { ... }
```

```
@Test
void teste2() { ... }
```

👉 Você faz **um único teste** rodando com vários valores.

1. Dependência necessária

Além do JUnit, adicione no pom.xml:

```
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter-params</artifactId>
  <version>5.10.0</version>
  <scope>test</scope>
</dependency>
```

2. Exemplo simples — @ValueSource

```
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.ValueSource;
import static org.junit.jupiter.api.Assertions.*;

class CalculadoraTest {

    @ParameterizedTest
    @ValueSource(ints = {1, 2, 3, 4})
    void deveTestarNumerosPositivos(int numero) {
        assertTrue(numero > 0);
    }
}
```

👉 Esse teste roda **4 vezes automaticamente**

3. Exemplo prático — somar

```
@ParameterizedTest
@CsvSource({
    "2,3,5",
    "10,5,15",
    "0,0,0"
})
void deveSomarCorretamente(int a, int b, int resultado) {
    Calculadora calc = new Calculadora();
    assertEquals(resultado, calc.somar(a, b));
}
```

4. Exemplo importante — testar exceção

```
@ParameterizedTest
@ValueSource(ints = {0, -1})
void deveLancarExcecaoQuandoDivisorInvalido(int divisor) {
    Calculadora calc = new Calculadora();

    assertThrows(IllegalArgumentException.class, () -> {
        calc.dividir(10, divisor);
    });
}
```

5. Usando @MethodSource (mais avançado)

Quando os dados são mais complexos:

```
import java.util.stream.Stream;
import org.junit.jupiter.params.provider.Arguments;

static Stream<Arguments> dadosSoma() {
    return Stream.of(
        Arguments.of(2, 3, 5),
        Arguments.of(4, 6, 10),
        Arguments.of(1, 1, 2)
    );
}
```

```

    );
}

@ParameterizedTest
@MethodSource("dadosSoma")
void deveSomarComMethodSource(int a, int b, int resultado) {
    Calculadora calc = new Calculadora();
    assertEquals(resultado, calc.somar(a, b));
}

```

6. Quando usar cada um

Tipo	Quando usar
@ValueSource	valores simples
@CsvSource	múltiplos parâmetros
@MethodSource	cenários complexos
@CsvFileSource	dados de arquivo